

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 05, Abgabe am 01.12.25 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Hausaufgabe 1 (15 Punkte):

In dieser Aufgabe geht es um eine kleine Datenbank über Furbys. Sie erhalten diese Datenbank als die ebenfalls bereitgestellte Datei `furby_datei.csv`, welche im Comma-separated-values-Format (kurz: CSV) vorliegt. Das bedeutet, dass jede Zeile genau einen Datensatz enthält. In unserem Fall ist ein Datensatz der Name eines Furbys, seine primäre Farbe, seine sekundäre Farbe, sein Accessoire (Tail oder Mane) und 10 weitere Einträge, die den Bestand in verschiedenen Geschäften angeben. Diese Werte sind aufsteigend sortiert. Die Daten innerhalb eines Datensatzes, also innerhalb einer Zeile, werden durch Kommas getrennt.

Eine der Zeilen ist z.B.

`Frog, Olive Green, Light Cream, Mane, 4, 4, 4, 6, 9, 11, 15, 16, 19, 20`

Sie sollen die Datenbank bearbeiten und analysieren, indem Sie die folgenden Funktionen implementieren und sie im Hauptprogramm nacheinander auf die Datenbank anwenden. Ihr Programm muss auch noch funktionieren, wenn der Datenbank neue Datensätze hinzugefügt werden.

- (a) (3 Punkte) Eine Funktion `csv_nach_zeilentupel`, welche als Eingabe einen Dateipfad (String) zu einer CSV-Datei erwartet und welche die Datenbank aus dieser Datei in eine Liste von Tupeln umwandelt. D.h. die Rückgabe soll eine Liste von Tupeln sein, wobei jedes Tupel einen der Datensätze enthält und die Tupel-Einträge alle vom Typ String sind. Achten Sie darauf überflüssige Leerzeichen und `\n` zu entfernen.

Hinweis: Verwenden Sie `x = open(datei_pfad, 'r').readlines()`, wobei `datei_pfad` der entgegengenommene Dateipfad ist. Nachdem diese Zeile ausgeführt wurde, enthält `x` eine Liste von Strings (einen String pro Zeile in der CSV-Datei). Sie können außerdem `tuple(liste)` verwenden um eine Liste `liste` in ein Tupel umzuwandeln.

Hinweis: Wenn Sie die Datei `furby_datei.csv` im gleichen Verzeichnis wie Ihr Programm abspeichern, dann lautet der Dateipfad `"furby_datei.csv"`. Dann fängt die Rückgabe von `csv_nach_zeilentupel("furby_datei.csv")` an mit:

```
[('Pink Flamingo', 'Light Pink', 'Cerise', 'Tail', '3', '6', '6', '12', '13', '13', '13', '14', '21', '22'),  
 ('Wizard', 'Black', 'Violet', 'Tail', '1', '4', '7', '9', '13', '16', '17', '18', '22', '22'), ...]
```

- (b) (2 Punkte) Eine Funktion `aufraeumen`, welche als Eingabe die Rückgabe aus (a), also die Datenbank, dargestellt als Liste von Tupeln, erwartet. Beide Spalten, die die Farbe angeben, sollen entfernt werden. Außerdem sollen die Bestandsangaben, die pro Datensatz angegeben sind, so umsortiert werden, dass sie statt in aufsteigender, in absteigender Reihenfolge angegeben sind. Ausgegeben werden soll die modifizierte Datenbank im Listen-Tupel-Format.

Hinweis: Wendet man `aufraeumen` auf die Liste aus dem vorherigen Hinweis an, so fängt die Rückgabe an mit:

```
[('Pink Flamingo', 'Tail', '22', '21', '14', '13', '13', '13', '12', '6', '6', '3'),
 ('Wizard', 'Tail', '22', '22', '18', '17', '16', '13', '9', '7', '4', '1'), ...
```

- (c) (2 Punkte) Eine Funktion `ident`, welche als Eingabe die Rückgabe aus (b), also die „aufgeräumte“ Datenbank, dargestellt als Liste von Tupeln, erwartet. Ihre Funktion soll die Datensätze so umordnen, dass Sie entsprechend dem Namen des Furbys alphabetisch sortiert sind. Anschließend soll jeder Datensatz als ersten Eintrag eine ID bekommen: die erste Zeile erhält die ID 1, die zweite Zeile die ID 2 usw. Ausgegeben werden soll die modifizierte Datenbank im Listen-Tupel-Format.

Hinweis: Wendet man `ident` auf die Liste aus dem vorherigen Hinweis an, so fängt die Rückgabe an mit:

```
[('1', 'Banana Peel', 'Mane', '17', '15', '13', '13', '10', '6', '6', '6', '1', '1'),
 ('2', 'Bandit Elephant', 'Mane', '22', '20', '16', '15', '13', '13', '13', '12', '5', '3'), ...
```

- (d) (3 Punkte) Eine Funktion `arithm_mittel`, welche als Eingabe die Rückgabe aus (c), also die „aufgeräumte“, sortierte Datenbank mit IDs, dargestellt als Liste von Tupeln, erwartet. Ihre Funktion soll die Datensatz-Einträge, die die Bestände angeben, in Integer-Werte umwandeln (bislang liegen diese als String vor). Anschließend soll jedem Datensatz als letzter Eintrag das arithmetische Mittel der Bestandszahlen hinzugefügt werden (als Float-Wert). Ausgegeben werden soll die modifizierte Datenbank im Listen-Tupel-Format.

Hinweis: Wendet man `arithm_mittel` auf die Liste aus dem vorherigen Hinweis an, so fängt die Rückgabe an mit:

```
[('1', 'Banana Peel', 'Mane', 17, 15, 13, 13, 10, 6, 6, 6, 1, 1, 8.8),
 ('2', 'Bandit Elephant', 'Mane', 22, 20, 16, 15, 13, 13, 13, 12, 5, 3, 13.2), ...
```

- (e) (5 Punkte) Eine Funktion `median`, welche als Eingabe die Rückgabe aus (d), also die vollständig modifizierte Datenbank, dargestellt als Liste von Tupeln, erwartet. Ihre Funktion soll für jedes Accessoire (Tail und Mane) den Median, über die zuvor berechneten arithmetischen Mittel, berechnen und in der Konsole ausgeben. Der Median beschreibt den Wert der, wenn man die Datenpunkte sortiert aufzählt, genau in der Mitte liegt, z.B. von 1, 2, 3, 9, 9 ist der Median 3 und von 1, 1, 2, 3, 9, 9 ist der Median 2.5. Zurückgegeben werden soll `None`.

Hinweis: Wendet man `median` auf die Rückgabe aus dem vorherigen Hinweis an, so soll in der Konsole folgendes ausgegeben werden:

```
Tail: 12.2
Mane: 10.45
```

Hausaufgabe 2 (5 Punkte):

Schreiben Sie eine Funktion `preimage`, welche drei Eingaben erwartet: `funktion`, `startintervall` und `zielintervall`. Hierbei ist `funktion` eine Funktion, welche einen Integer-Wert erwartet und einen Integer-Wert zurückgibt. Die Eingaben `startintervall` und `zielintervall` sind Paare von Integer-Werten, also Tupel der Länge zwei. Hierbei steht das Paar (i, j) für das mathematische Ganzzahl-Intervall $[i, j]$, also die Werte $i, i + 1, \dots, j$.

Die Funktion soll eine Liste von Listen zurückgeben, wobei jede der inneren Listen für einen Funktionswert aus dem Zielintervall steht. Die inneren Listen sollen jeweils die Werte aus dem Startintervall enthalten, welche unter der eingegebenen Funktion `funktion` auf den entsprechenden Zielwert abbilden. Sie dürfen davon ausgehen, dass die Eingaben wie beschrieben sind.

Hinweis: Wenn für die Integer-Werte $i > j$ gilt, dann ist $[i, j]$ das leere Intervall.

Beispiele: Seien die folgenden beiden Funktionen im Programm definiert:

```
def g (zahl):  
    return zahl+1  
  
def h (zahl):  
    return 1
```

Dann sollen die folgenden Funktionsaufrufe von `preimage` die angegebenen Listen zurückgeben:

- `preimage(g, (-2,2), (-2,2))` → `[[ ], [-2], [-1], [0], [1]]`
- `preimage(g, (6,10), (9,13))` → `[[8], [9], [10], [ ], [ ]]`
- `preimage(h, (12,14), (0,3))` → `[[ ], [12,13,14], [ ], [ ]]`
- `preimage(h, (12,14), (3,2))` → `[ ]`

Präsenzaufgaben

Präsenzaufgabe 1:

Eine Ganzzahl-Matrix ist eine Liste deren Elemente Integer-Listen der gleichen Länge sind. Man bezeichnet eine Ganzzahl-Matrix der Länge k deren Elemente Listen der Länge l sind als (k, l) -Ganzzahl-Matrix. In diesem Fall bezeichnet k die Anzahl von Zeilen und l die Anzahl von Spalten der (k, l) -Ganzzahl-Matrix.

Zum Beispiel ist `[[10, 3, 29, 4], [4, 77, 191, 2], [85, 9, 2, 37]]` eine Ganzzahl-Matrix, während `[[13, 3.75], [4.0, 2.23], [80, 3]]` und `[[13, 3], [4], [80, 3]]` keine sind.

- (a) Schreiben Sie eine Funktion `matrix_mult`, welche zwei Listen M und N entgegennimmt und diese, sofern möglich, als Ganzzahl-Matrizen interpretiert und $M \cdot N$ berechnet. Ist die Multiplikation möglich, soll das Ergebnis sowohl zurückgegeben, als auch in geeigneter Darstellung auf der Konsole ausgegeben werden. Falls eine Multiplikation nicht möglich ist, soll auf der Konsole "Keine Multiplikation möglich" ausgegeben und `None` zurückgegeben werden.

Definieren Sie dazu die folgenden Hilfsfunktionen:

- Eine Funktion `ist_integer_matrix`, welche eine Liste M entgegennimmt und prüft, ob M eine Ganzzahl-Matrix ist. Falls ja sollen die Dimensionen (k, l) als Tupel zurückgegeben werden, ansonsten `False`.
- Eine Funktion `ist_kompatibel`, welche zwei Listen M und N entgegennimmt und prüft, ob es sich um Ganzzahl-Matrizen handelt, deren Dimensionen geeignet sind, um $M \cdot N$ berechnen zu können. Falls ja soll `True` zurückgegeben werden, ansonsten `False`.

Hinweis: Damit $M \cdot N$ berechnet werden kann, müssen M und N Ganzzahl-Matrizen sein und die Anzahl der Spalten in M muss mit der Anzahl der Zeilen in N übereinstimmen.

- Eine Funktion `print_matrix`, welche eine Liste M entgegennimmt und diese in Matrixdarstellung auf der Konsole ausgibt, sofern es sich bei M um eine Ganzzahl-Matrix handelt. Andernfalls soll die Ausgabe "Keine Matrix" auf der Konsole erfolgen. Der Rückgabewert soll in jedem Fall `None` sein.

Die Matrixdarstellung der Matrix `[[1, 2], [3, 4], [5, 6]]` sieht wie folgt aus:

```
1 2
3 4
5 6
```

Es genügt, wenn Ihre Darstellung für Zahlen gleicher Stelligkeit richtig ausgerichtet ist.

- (b) Schreiben Sie nun eine Funktion `generate_matrix`, welche positive Zahlen k und l , sowie ganze Zahlen i und j entgegennimmt und eine (k, l) -Ganzzahl-Matrix mit zufälligen ganzzahligen Einträgen zwischen i und j erzeugt und diese zurückgibt.

Testen Sie nun Ihre Funktion `matrix_mult` mit zufälligen Matrizen verschiedener Größen und selbst angelegten Matrizen.

Benutzen Sie den Befehl `random.randint(i, j)` und importieren Sie zuvor das Modul `random`.

Hinweis 1: Sie dürfen davon ausgehen, dass alle Eingaben vom korrekten Datentyp sind. Für Matrizen, bei denen eine der Dimensionen 0 ist, dürfen sich Ihre Funktionen beliebig verhalten.

Hinweis 2: Machen Sie sich zuvor an der Tafel klar, wie Matrixmultiplikation funktioniert und berechnen Sie einige Beispiele, die Sie zum Testen Ihrer Funktionen verwenden können.

Präsenzaufgabe 2:

Schreiben Sie eine Funktion `dec`, welche die Vorgängerfunktion für Integer realisiert und eine Funktion `inc`, welche die Nachfolgerfunktion für Integer realisiert. Lösen Sie die folgenden Aufgaben mittels Rekursion und verwenden Sie dabei keine vordefinierten arithmetischen Operatoren wie `+`, `-`, `*`, `**`, sondern benutzen Sie `dec`, `inc` und Ihre anderen Funktionen.

- (a) Schreiben Sie eine Funktion `add`, welche als Eingabe zwei Integer $n, m \geq 0$ erwartet und das Ergebnis von $n + m$ zurückgibt.
- (b) Schreiben Sie eine Funktion `mult`, welche als Eingabe zwei Integer $n, m \geq 0$ erwartet und das Ergebnis von $n \cdot m$ zurückgibt.
- (c) Schreiben Sie eine Funktion `exp`, welche als Eingabe zwei Integer $n, m \geq 0$ erwartet und das Ergebnis von n^m zurückgibt.
- (d) Schreiben Sie eine Funktion `sq_row`, welche als Eingabe einen Integer $n \geq 0$ erwartet und das Ergebnis von $\sum_{i=0}^n i^2$ zurückgibt.